Linux Basic Commands

A Practical Reference for MLOps Coursework

Author: Dr. Santosh Kumar

Date: 23rd Aug 2026

This guide covers the core Linux command-line skills that come up repeatedly in MLOps work: navigating a filesystem, managing files and permissions, controlling processes, installing packages, working with environment variables, handling networking and remote machines, monitoring system resources, and using shell tools that support data pipelines, training jobs, and deployment scripts. Each command includes its syntax, a short explanation, and a practical example.

Contents

1. File & Directory Navigation
2. File & Directory Management
3. Viewing and Editing File Contents
4. File Permissions & Ownership
5. Process & Job Management
6. System Monitoring & Resource Usage
7. Disk Usage & Storage

1. File & Directory Navigation

pwd — Print Working Directory

Displays the absolute path of the directory you are currently in. Useful for confirming your location before running scripts that use relative paths (a common source of bugs in training pipelines).

ls — List Directory Contents

Lists files and folders in a directory. Common flags: `-l` (long/detailed format), `-a` (show hidden files), `-h` (human-readable sizes), `-t` (sort by modification time).

Example:

```
ls -lah /home/user/project
ls -lt models/ # newest model checkpoints first
```

cd — Change Directory

Moves you into a different directory. `cd ..` moves up one level, `cd ~` or `cd` alone goes to your home directory, and `cd -` returns to the previous directory.

Example:

```
cd /mnt/data/datasets
cd ..
cd ~/mlops-project
```

find — Search for Files and Directories

Recursively searches a directory tree for files matching a name, type, size, or modification time. Extremely useful for locating stray checkpoint files, logs, or config files in large project directories.

Example:

```
find . -name "*.pkl"
find /data -type f -size +500M
find . -mtime -1 # modified in last 1 day
```

which / whereis — Locate a Command

`which` shows the full path of the executable that would run for a given command (useful for checking which Python or pip is active in a virtual environment). `whereis` also shows man pages and source locations.

Example:

```
which python3
which pip
```

2. File & Directory Management

mkdir — Make Directory

Creates a new directory. Use `-p` to create nested directories in one command (and to avoid an error if the directory already exists).

Example:

```
mkdir models
mkdir -p experiments/run_01/checkpoints
```

touch — Create an Empty File / Update Timestamp

Creates a new empty file if it doesn't exist, or updates its last-modified timestamp if it does. Often used to create placeholder files like `_init_.py` or `.gitkeep`.

Example:

```
touch requirements.txt
touch src/_init_.py
```

cp — Copy Files or Directories

Copies files or directories. Use `-r` (recursive) for directories and `-v` for verbose output showing what was copied.

Example:

```
cp config.yaml config_backup.yaml
cp -r dataset/
/mnt/backup/dataset/
```

mv — Move or Rename Files

Moves a file/directory to a new location, or renames it if the destination is in the same directory.

Example:

```
mv model_v1.pt model_v2.pt
mv results.csv reports/results_2026.csv
```

rm — Remove Files or Directories

Deletes files. Use `-r` to delete directories recursively and `-f` to force deletion without prompts. Because this command is irreversible, it is worth double-checking the path before running it, especially with wildcards.

Example:

```
rm old_log.txt
rm -r _pycache_/
rm -rf tmp_checkpoints/
```

Note: rm -rf permanently deletes data with no recycle bin. Always verify the target path (e.g. with ls first) before running it, particularly in automated scripts.

3. Viewing and Editing File Contents

cat — Concatenate and Display File Contents

Prints the entire contents of a file to the terminal. Good for short files like configs or small logs; also used to quickly combine multiple files.

Example:

```
cat config.yaml
cat file1.txt file2.txt > combined.txt
```

less / more — Page Through a File

Displays file contents one screen at a time, which is far more practical than `cat` for large log files. Navigate with arrow keys, search with `/pattern`, and quit with `q`.

Example:

```
less training.log
```

head / tail — View the Start or End of a File

`head` shows the first lines of a file (default 10), `tail` shows the last lines. `tail -f` continuously streams new lines as they are written — ideal for watching a training log in real time.

Example:

```
head -n 20 data.csv
tail -f training.log # live-follow a running job's output
```

nano / vim — Terminal Text Editors

Edit files directly in the terminal, which matters on remote servers or containers with no graphical interface. `nano` is beginner-friendly (on-screen shortcuts); `vim` is more powerful but has a steeper learning curve.

Example:

```
nano train_config.yaml
vim Dockerfile
```

grep — Search Text Inside Files

Searches for lines matching a pattern. Common flags: `-i` (case-insensitive), `-r` (recursive search through a directory), `-n` (show line numbers).

Example:

```
grep "ERROR" training.log
grep -rn "learning_rate" ./src
```

4. File Permissions & Ownership

chmod — Change File Permissions

Controls read (r), write (w), and execute (x) permissions for the owner, group, and others. Permissions can be set numerically (e.g. 755) or symbolically (e.g. u+x). Shell scripts must have execute permission before they can be run directly.

Example:

```
chmod +x run_training.sh
chmod 644 config.yaml
./run_training.sh
```

chown — Change File Owner

Changes the user (and optionally group) that owns a file or directory. Commonly needed when files created inside a Docker container end up owned by root on the host.

Example:

```
sudo chown user:user model.pt
sudo chown -R user:user
./outputs
```

sudo — Execute as Superuser

Runs a single command with administrator (root) privileges, required for tasks like installing system packages or changing ownership of protected files.

Example:

```
sudo apt update
sudo systemctl restart docker
```

5. Process & Job Management

ps — List Running Processes

Displays currently running processes. `ps aux` shows all processes system-wide with CPU/memory usage — useful for finding a runaway training script.

Example:

```
ps aux | grep python
```

top / htop — Live Process Monitor

Shows a real-time, continuously updating view of running processes, CPU load, and memory usage. `htop` is an enhanced, more readable version (may need to be installed separately). Press `q` to exit.

Example:

```
top
htop
```

kill / kill -9 — Terminate a Process

Sends a signal to stop a process using its Process ID (PID), found via `ps`. A plain `kill` requests a graceful shutdown; `kill -9` forces immediate termination.

Example:

```
kill 12345
kill -9 12345
```

& and nohup — Run Jobs in the Background

Appending `&` runs a command in the background so the terminal stays free. `nohup` additionally lets the process keep running after you log out or close the SSH session, writing output to a file.

Example:

```
python train.py &
nohup python train.py > train.log 2>&1 &
```

jobs / fg / bg — Manage Background Jobs

`jobs` lists background jobs in the current shell, `fg` brings one to the foreground, and `bg` resumes a paused job in the background.

Example:

```
jobs
fg %1
```

6. System Monitoring & Resource Usage

df — Disk Free Space

Shows available and used disk space across mounted filesystems. `-h` gives human-readable sizes (GB/MB), important for checking whether there's room for a new dataset or model checkpoint.

Example:

```
df -h
```

du — Disk Usage

Shows how much space a file or directory is using. `du -sh` summarizes total size; combine with `sort` to find the largest folders.

Example:

```
du -sh datasets/
du -h --max-depth=1 . | sort -h
```

free — Memory Usage

Displays total, used, and free RAM and swap space. `-h` shows values in human-readable units. Useful for diagnosing out-of-memory errors during training.

Example:

```
free -h
```

nvidia-smi — GPU Status

Displays GPU utilization, memory usage, temperature, and running processes on NVIDIA GPUs. Essential in MLOps for checking whether a training job is actually using the GPU and how much VRAM is free.

Example:

```
nvidia-smi
watch -n 1 nvidia-smi # refresh every second
```

7. Networking & Remote Access

ssh — Secure Shell (Remote Login)

Connects to a remote machine (a training server, cloud VM, or container host) over an encrypted connection. Requires the remote host's address and typically a key or password.

Example:

```
ssh user@192.168.1.10
ssh -i mykey.pem ubuntu@ec2-instance-address
```

scp — Secure Copy Over SSH

Copies files or directories between a local machine and a remote one over SSH. Use `-r` for directories.

Example:

```
scp model.pt user@server:/home/user/models/
scp -r ./dataset user@server:/data/
```

curl / wget — Download Files or Query APIs

`curl` and `wget` fetch content from a URL, useful for downloading datasets/model weights or testing a deployed inference API endpoint.

Example:

```
wget https://example.com/dataset.zip
curl -X POST http://localhost:8000/predict -d '{"x":1}'
```

ping — Test Network Connectivity

Checks whether a remote host is reachable and measures round-trip latency. A quick first step when a remote server or API seems unresponsive.

Example:

```
ping -c 4 8.8.8.8
```

8. Archiving, Compression & Transfer

tar — Archive Files

Bundles multiple files/directories into a single archive, commonly combined with compression. `-c` create, `-x` extract, `-z` gzip compression, `-v` verbose, `-f` specify filename.

Example:

```
tar -czvf dataset.tar.gz dataset/
tar -xzvf dataset.tar.gz
```

zip / unzip — Zip Archives

Creates or extracts `.zip` archives, a widely used cross-platform format for sharing datasets, code, or model artifacts.

Example:

```
zip -r project.zip project/
unzip project.zip -d
./extracted
```

rsync — Efficient File Synchronization

Synchronizes files/directories between locations (local or remote over SSH), only transferring the parts that changed. Much faster than `scp` for large datasets that are updated incrementally.

Example:

```
rsync -avz ./dataset/ user@server:/data/dataset/
```

9. Text Processing & Piping

| — Pipe

Sends the output of one command as input to another, allowing small commands to be chained into powerful one-liners for filtering logs, data, or process lists.

Example:

```
cat training.log | grep "loss" | tail -20
```

> and >> — Redirection

> writes command output to a file, overwriting it; `>>` appends to the end of a file instead.

Example:

```
python train.py > train.log
echo "run complete" >> train.log
```

wc — Word/Line/Character Count

Counts lines, words, or characters. `wc -l` is a fast way to check how many rows are in a large CSV file without opening it.

Example:

```
wc -l dataset.csv
```

sort / uniq — Sort and Deduplicate Lines

`sort` orders lines alphabetically or numerically (`-n`); `uniq` removes or counts adjacent duplicate lines (often used together, since uniq only works on sorted input).

Example:

```
sort scores.txt | uniq -c
sort -nr file_sizes.txt | head -5
```

awk / sed — Pattern-Based Text Processing

`awk` processes structured text column by column (e.g. extracting a field from a CSV or log line); `sed` performs search-and-replace or other line-by-line edits directly from the command line.

Example:

```
awk -F',' '{print $2}' data.csv
sed -i 's/old_value/new_value/g' config.yaml
```

10. Disk Usage & Storage Housekeeping

df -h and du -sh — Quick Recap

As covered in Section 6, these two commands are the first stop when a job fails due to insufficient disk space: `df -h` shows free space per filesystem, and `du -sh *` reveals which folders are the biggest space users.

watch — Repeat a Command Automatically

Reruns a given command at a fixed interval, useful for monitoring disk space, GPU usage, or a log directory while a long job is running.

Example:

```
watch -n 5 df -h
```

11. Git Basics for MLOps

git clone / pull / push — Sync with a Remote Repository

`clone` downloads a repository for the first time; `pull` fetches and merges the latest changes; `push` uploads local commits to the remote (e.g. GitHub, GitLab), which typically also triggers CI/CD pipelines in MLOps setups.

Example:

```
git clone
https://github.com/user/project.git git pull
origin main
git push origin main
```

git add / commit — Save Changes

`git add` stages changed files for the next commit; `git commit` saves a snapshot of the staged changes with a descriptive message, forming the version history of code and configs.

Example:

```
git add train.py requirements.txt
git commit -m "Add learning rate scheduler"
```

git status / log / diff — Inspect Repository State

`status` shows changed/staged files, `log` shows commit history, and `diff` shows line-by-line differences — all useful for reviewing what changed before committing or deploying.

Example:

```
git status
git log --oneline -10
git diff train.py
```

git branch / checkout — Work with Branches

Branches let you develop or experiment (e.g. a new model architecture) without affecting the main codebase. `checkout -b` creates and switches to a new branch in one step.

Example:

```
git checkout -b feature/new-model
git checkout main
```

12. Useful MLOps-Specific Command Patterns

Running a Training Job in the Background with Logging

Combines backgrounding, output redirection, and process independence so a long training run survives an SSH disconnect and its output is saved for later review.

Example:

```
nohup python train.py --epochs 50 > logs/train_$(date +%F).log 2>&1 &
```

Watching GPU and Memory Together

Running two monitors side by side (in separate terminal panes or a tmux split) gives a live picture of whether a job is GPU-bound or memory-bound.

Example:

```
watch -n 1 nvidia-smi
watch -n 1 free -h
```

tmux / screen — Persistent Terminal Sessions

Creates a terminal session that keeps running on the server even after you disconnect, and lets you reattach later. This is the standard alternative to `nohup` for long MLOps jobs that need interactive monitoring.

Example:

```
tmux new -s training
# ... run your job ...
# detach: Ctrl+b then
d tmux attach -t
training
```

Finding and Cleaning Up Large Checkpoint Files

A common storage-hygiene routine: locate the largest files in a checkpoints directory and remove the ones no longer needed before disk space runs out.

Example:

```
find ./checkpoints -name "*.ckpt" -printf '%s %p\n' | sort -nr | head -10
rm ./checkpoints/epoch_01.ckpt
```

Checking a Service or Container Port

Verifies that a model-serving process (e.g. FastAPI, TorchServe, MLflow) is actually listening on the expected port before debugging further upstream.

Example:

```
sudo lsof -i :8000
curl http://localhost:8000/health
```

Tip: Run `man <command>` *(e.g.* `man grep`*) any time you need the full manual and flag list for a command directly in the terminal.*